Python + MCP

🛠️ Dec 16: Building MCP servers with FastMCP
☁️ Dec 17: Deploying MCP servers to the cloud
👤 Dec 18: Authentication for MCP servers

🔗 Register at aka.ms/PythonMCP/series

Microsoft

Python + MCP

🛠️ Building MCP servers with FastMCP

aka.ms/pythonmcp/slides/servers

Pamela Fox

Python Cloud Advocate

www.pamelafox.org

Today we'll cover…

- What is MCP?
- Building MCP servers in Python with FastMCP
- Using local MCP servers in GitHub Copilot in VS Code
- Debugging, inspecting, and observing local MCP servers
- Using AI agents with local MCP servers

Want to follow along?

1. Open this GitHub repository in GitHub Codespaces:

aka.ms/python-mcp-demos

2. Follow instructions in README:

Setup

1. Install dependencies using `uv`:

```
uv sync
```

2. Copy `.env-sample` to `.env` and configure your environment variables:

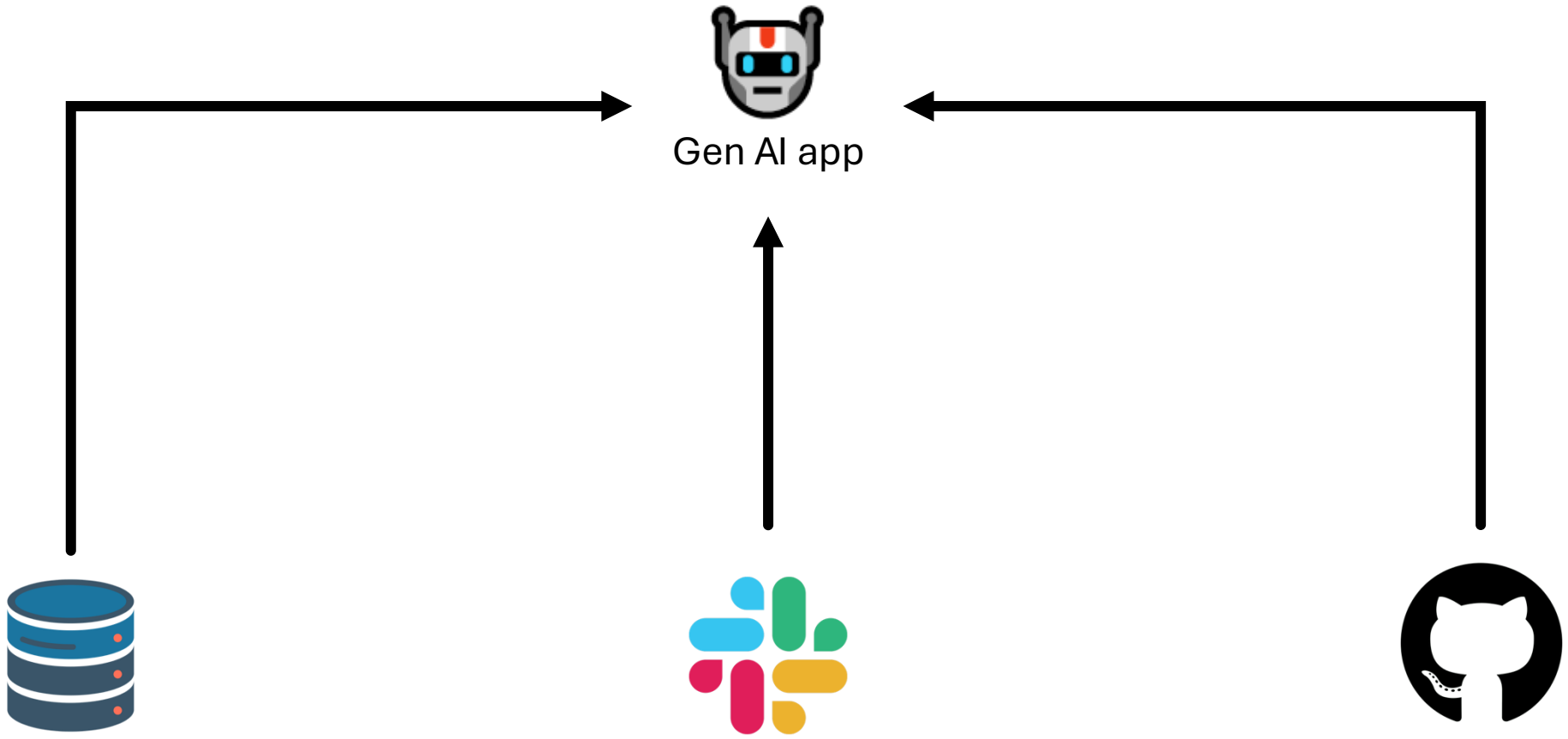
```
cp .env-sample .env
```

3. Edit `.env` with your API credentials. Choose one of the following providers by setting `API_HOST`:
   - `github` - GitHub Models (requires `GITHUB_TOKEN`)
   - `azure` - Azure OpenAI (requires Azure credentials)

What is MCP?

Before MCP

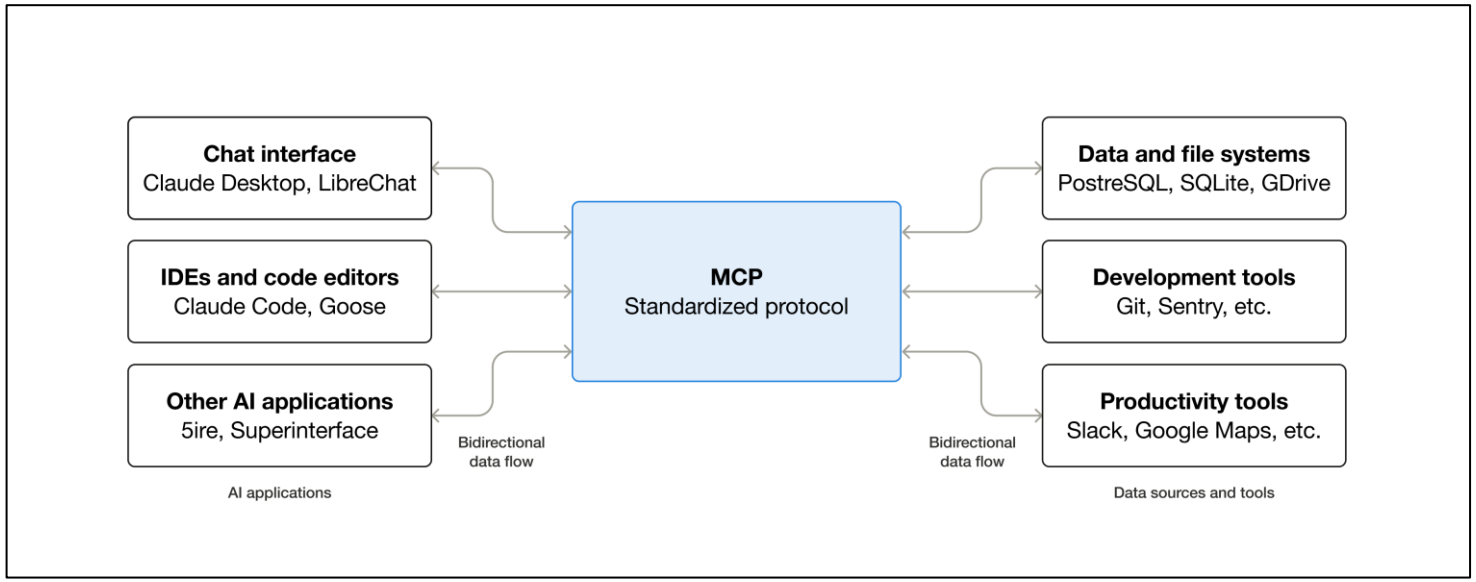
Every integration with your AI app had to be done individually:

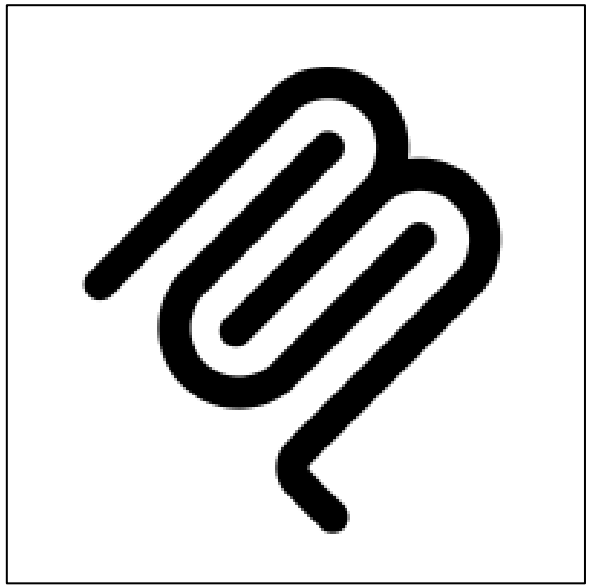


Model Context Protocol

https://modelcontextprotocol.io/

An **open protocol** created by Anthropic that **standardizes** the interaction between LLMs and **external tools**, **data sources**, and **applications**.

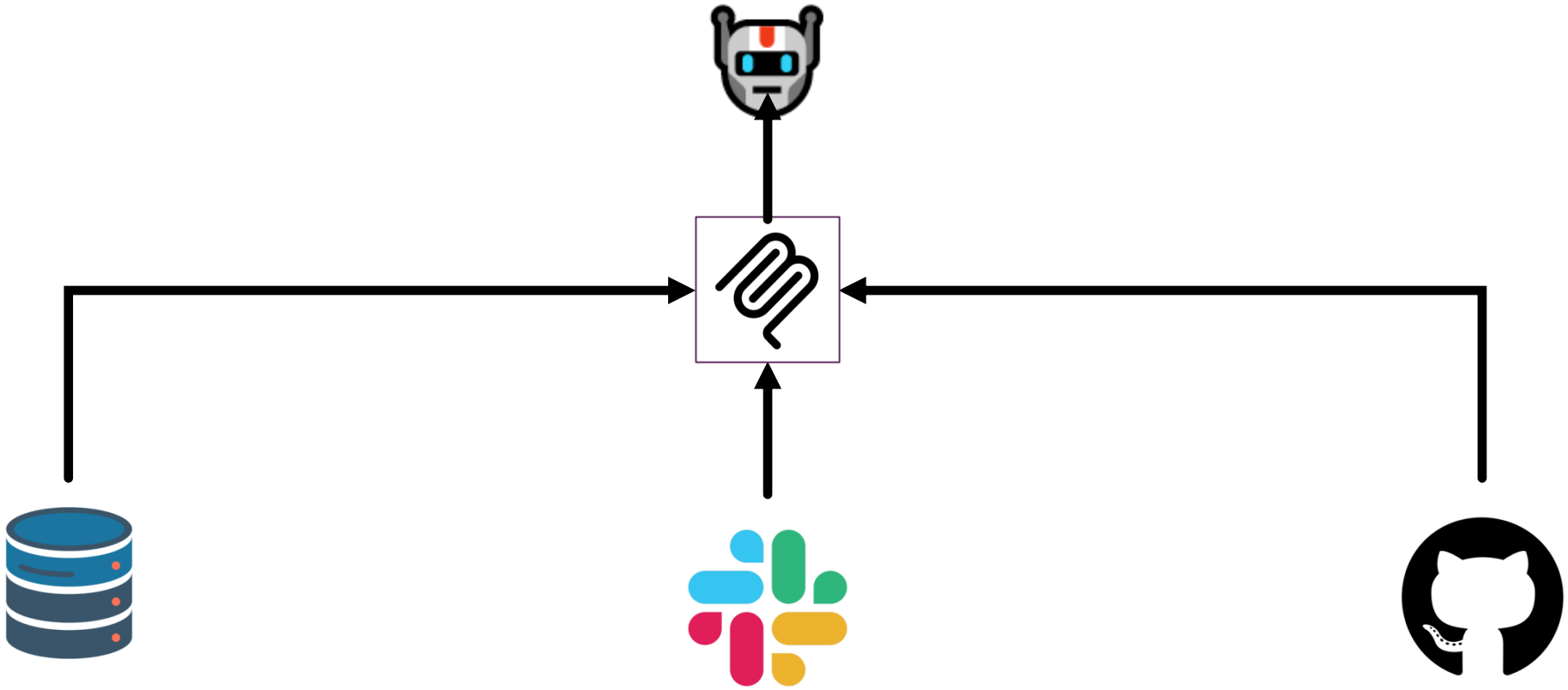




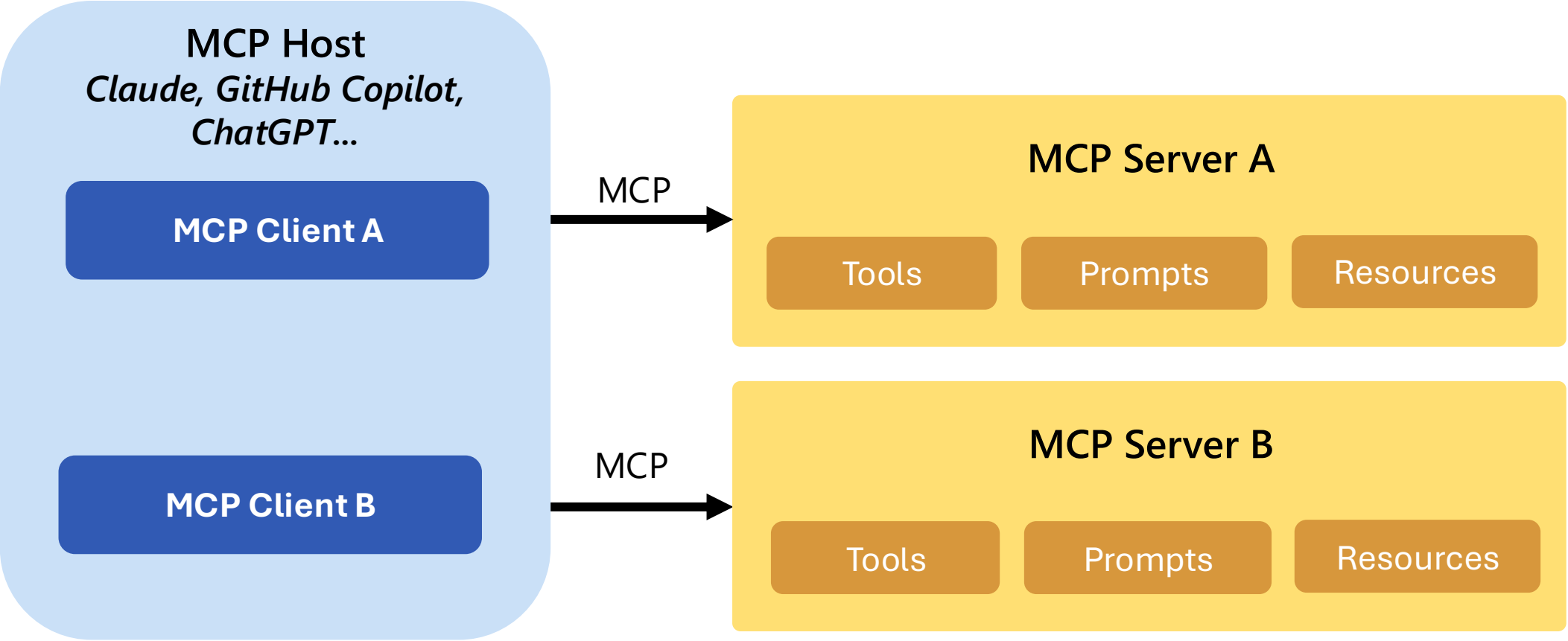
🎉 As of Dec. 2025, MCP is part of **Agentic AI Foundation** (**Linux Foundation**).

After MCP

When using MCP, the app **no longer needs to manage each integration separately**, since everything **goes through a standard protocol**.



Overview of MCP architecture



https://modelcontextprotocol.io/docs/learn/architecture

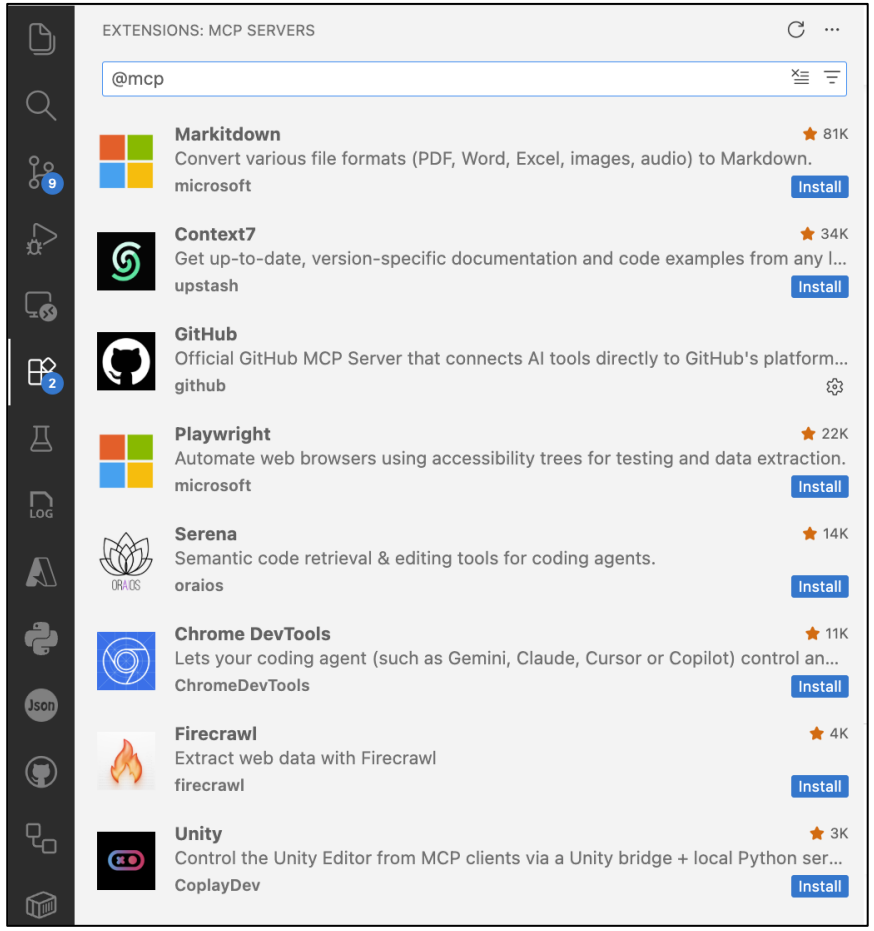
MCP clients

Client	Resources	Prompts	Tools	Discovery	Sampling	Roots	Elicitation
5ire	❌	❌	✅	?	❌	❌	?
AgentAI	❌	❌	✅	?	❌	❌	?
AgenticFlow	✅	✅	✅	✅	❌	❌	?
AIQL TUUI	✅	✅	✅	✅	✅	❌	✅
Amazon Q CLI	❌	✅	✅	?	❌	❌	?
Amazon Q IDE	❌	❌	✅	❌	❌	❌	?
Amp	✅	✅	✅	❌	✅	❌	?
Apify MCP Tester	❌	❌	✅	✅	❌	❌	?
Augment Code	❌	❌	✅	❌	❌	❌	?
BeeAI Framework	❌	❌	✅	❌	❌	❌	?
BoltAI	❌	❌	✅	?	❌	❌	?
Call Chirp	❌	✅	✅	❌	❌	❌	?
Chatbox	❌	❌	✅	❌	❌	❌	❌
ChatFrame	❌	❌	✅	❌	❌	❌	❌
ChatGPT	❌	❌	✅	❌	❌	❌	?
ChatWise	❌	❌	✅	❌	❌	❌	?

https://modelcontextprotocol.io/clients#feature-support-matrix

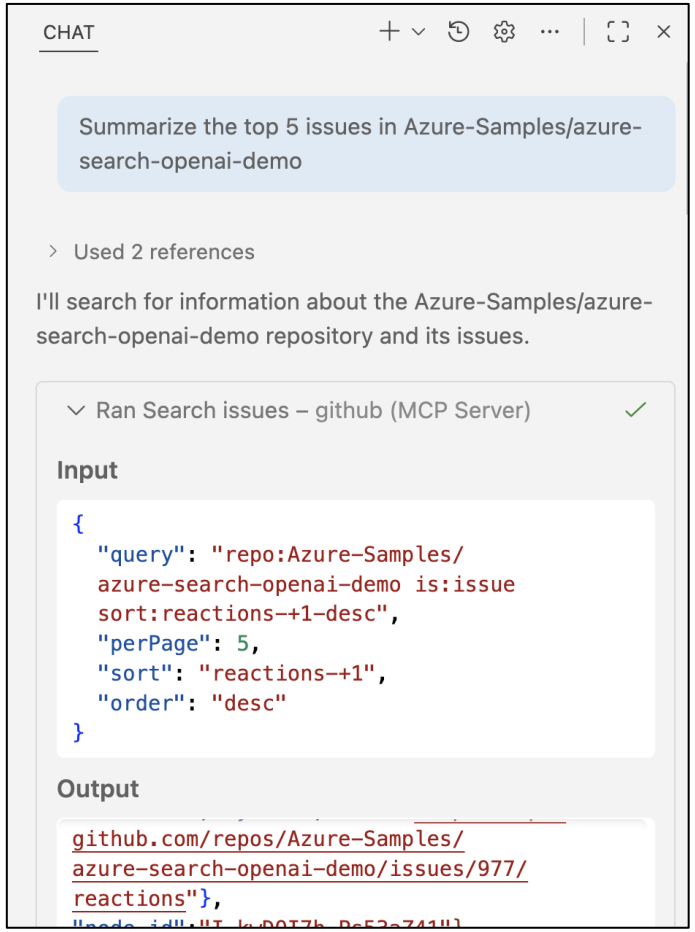
VS Code: Compliant MCP client

VS Code (+ GitHub Copilot) supports the complete [Model Context Protocol specification](#).



Install from extensions marketplace ←

Use from Copilot →



MCP server components

Function	Explanation	Examples	Who controls?
Tools	Functions your LLM can call and decide when to use, depending on the user's request. Tools can write to databases, call external APIs, modify files, or trigger other logic.	• Search for flights • Send messages • Create calendar events	Model
Resources	Passive data sources that provide read-only access for context (e.g., file contents, database schemas, or API documentation).	• Retrieve documents • Access knowledge bases • Read calendars	Application
Prompts	Pre-built instruction templates that tell the model how to work with specific tools and resources.	• Plan a vacation • Summarize meetings • Draft an email	User

https://modelcontextprotocol.io/docs/learn/server-concepts

Build an MCP server in Python

Python MCP frameworks

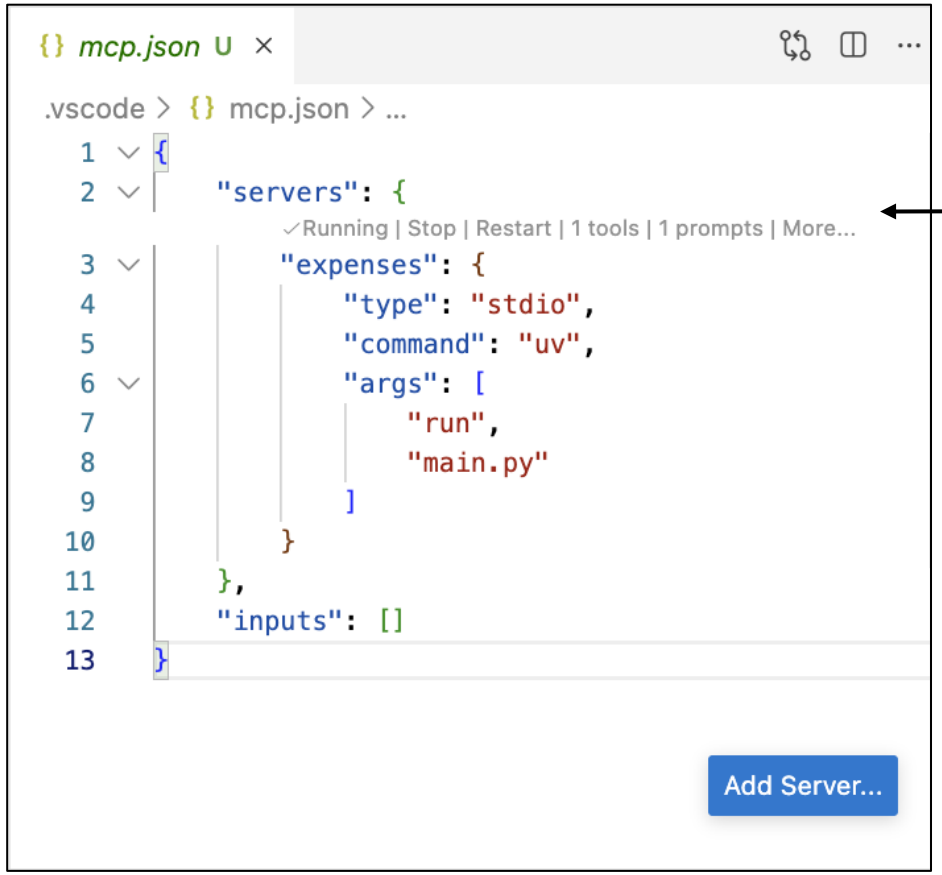
Framework	Description
python-sdk	Official Python SDK for MCP. Lets you build both clients and servers fully compatible with the protocol specification. Handles transport (stdio, SSE, HTTP), message cycles, tools, resources, and prompts.
fastmcp	Framework built on top of the official SDK. Adds production-focused features like server composition, proxying, OpenAPI/FastAPI generation, enterprise authentication (Google, GitHub, Azure, Auth0, etc.), deployment utilities, and client libraries.

Why use FastMCP?

Feature	Description
Pythonic decorators	Define tools, prompts, and resources with @mcp.tool, @mcp.prompt, @mcp.resource — simple and readable.
Type-hints	Uses Python type hints to automatically generate input/output schemas for LLMs.
Async	Supports async operations as well as standard (sync) functions.
Annotations and metadata	Mark tools as read-only, idempotent, or destructive for clearer behavior.
Multiple transports	Run the server over stdio or HTTP without changing your code.

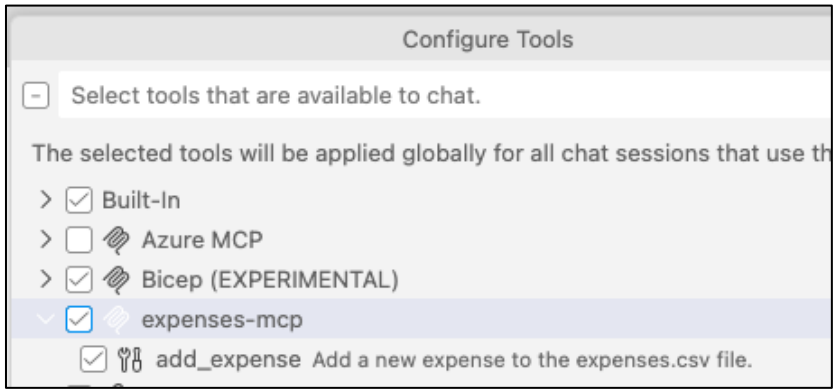
Running local FastMCP server in VS Code

1 Configure server in `.vscode/mcp.json`:



2 Start the server

3 Enable the server in Copilot Chat tools:



Tool with FastMCP

```python
@mcp.tool
async def add_expense(
        date: Annotated[date, "Date of the expense in YYYY-MM-DD format"],
        amount: Annotated[float, "Positive numeric amount of the expense"],
        category: Annotated[Category, "Category label"],
        description: Annotated[str, "Human-readable description of expense"],
        payment_method: Annotated[PaymentMethod, "Payment method used"],
):
  # Implementation ...

  return f"Added expense: ${amount} for {description} on {date_iso}"
```

Full example: basic_mcp_stdio.py

To trigger tool call in VS Code, send message in chat asking to log an expense.

Resource with FastMCP

```
@mcp.resource("resource://expenses")
async def get_expenses_data():
    csv_content = f"Expense data ({len(expenses_data)} entries):\n\n"
    for expense in expenses_data:
      csv_content += (
          f"Date: {expense['date']}, "
          f"Amount: ${expense['amount']}, "
          f"Category: {expense['category']}, "
          f"Description: {expense['description']}, "
          f"Payment: {expense['payment_method']}\n"
      )
 return csv_content
```

Full example: basic_mcp_stdio.py

To select resource in VS Code, use "Add Context..." button in chat

Prompt with FastMCP

```
@mcp.prompt
def analyze_spending_prompt(
  category: str | None = None,
  start_date: str | None = None,
  end_date: str | None = None) -> str:
  # Build filters
  return f"""Please analyze my spending patterns {filter_text} and provide:
    1. Total spending breakdown by category
    2. Average daily/weekly spending
    3. Most expensive single transaction
    4. Payment method distribution"""
```

Full example: basic_mcp_stdio.py

To use prompt in VS Code, type "/" and select prompt from options

MCP transports: STDIO vs. HTTP

	STDIO	HTTP (Streamable)
Connection mode	Local communication via stdin/stdout	Network communication over the HTTP protocol
Server startup	Launched on demand by the client (e.g., VS Code)	Runs as a service accessible through a URL
Typical use	Local development, CLI tools, single-user apps	Production, remote access, multiple simultaneous clients
Advantages	Simple, no network setup, ideal for quick testing	Scalable, supports bidirectional streaming, easy web integration
Limitations	One client per process, no network connection	Requires network and host/port configuration

https://gofastmcp.com/deployment/running-server

Serve FastMCP servers over HTTP

Specify HTTP transport:

```
mcp.run(transport="http", host="0.0.0.0", port=8000)
```

Start the server:

```
uv run basic_mcp_http.py
```

Configure the HTTP server in `mcp.json`:

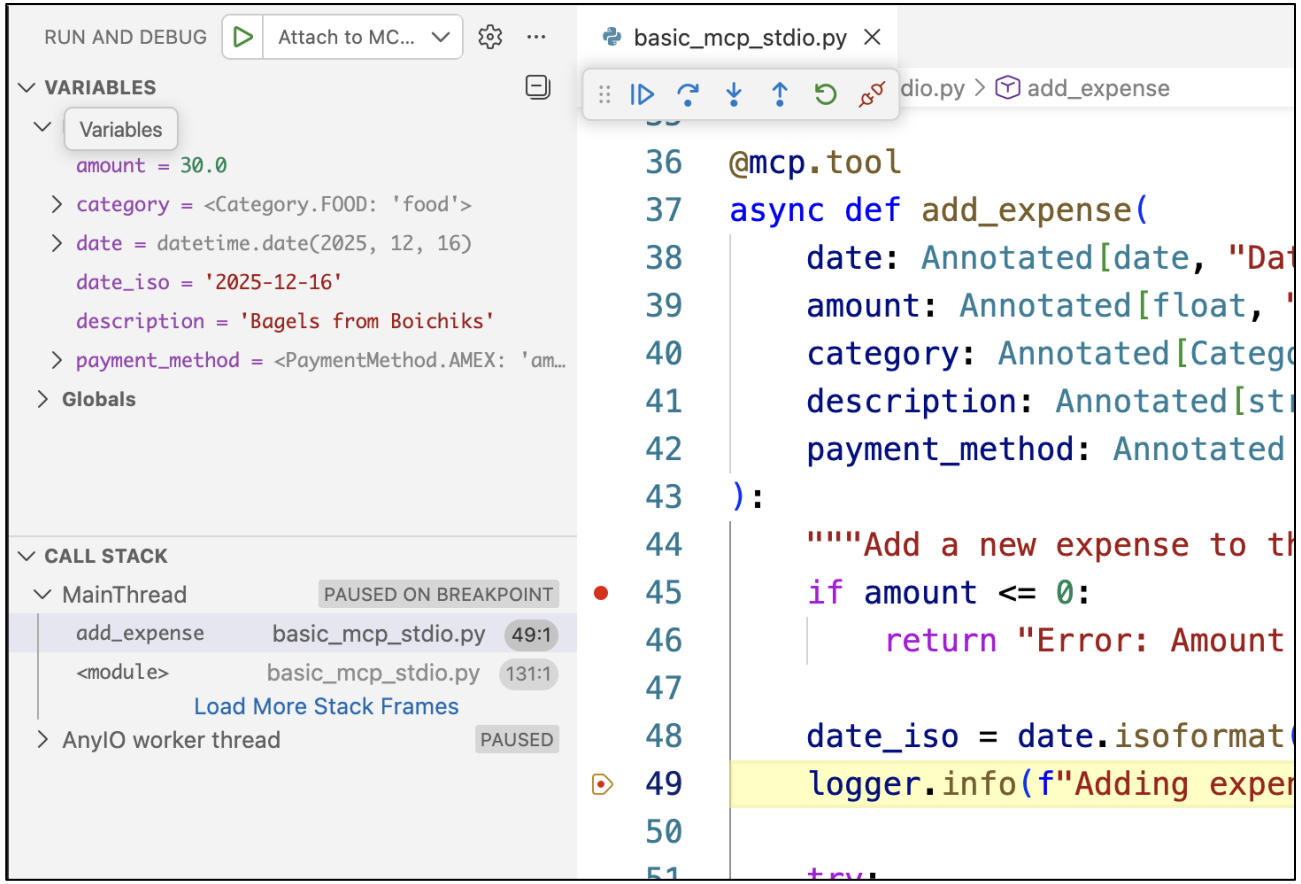
```
"expenses-mcp-http": {
  "type": "http",
  "url": "http://localhost:8000/mcp"
}
```

Development tips for MCP servers

Breakpoint debugging in VS Code

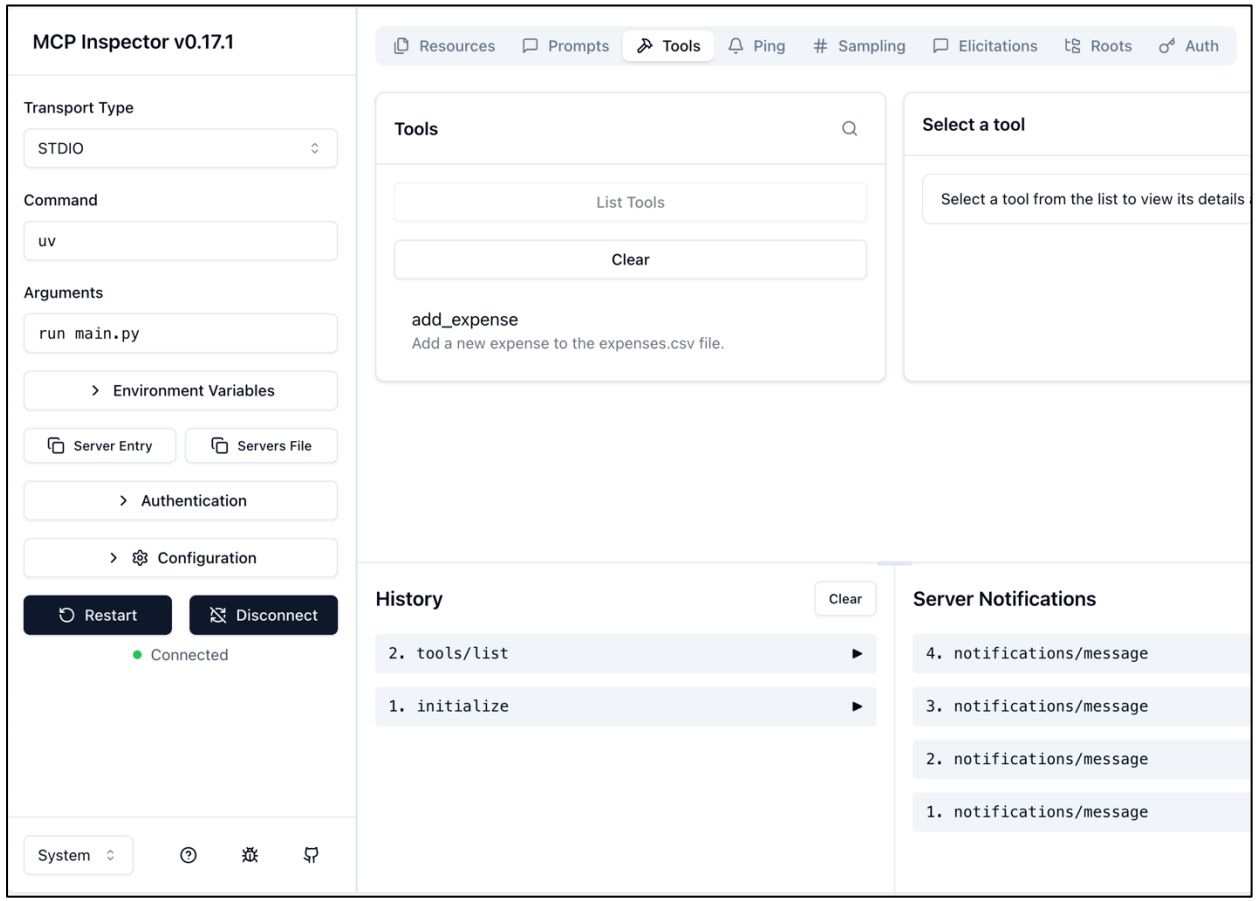
mcp.json:

```
"expenses-mcp-debug": {
  "type": "stdio",
  "command": "uv",
  "cwd": "${workspaceFolder}",
  "args": [
        "run",
        "--",
        "python",
        "-m",
        "debugpy",
        "--listen",
        "0.0.0.0:5678",
        "main.py"
  ]
}
```



code.visualstudio.com/docs/copilot/customization/mcp-servers#_debug-an-mcp-server

MCP Inspector



The MCP inspector is a developer tool for testing and debugging MCP servers.

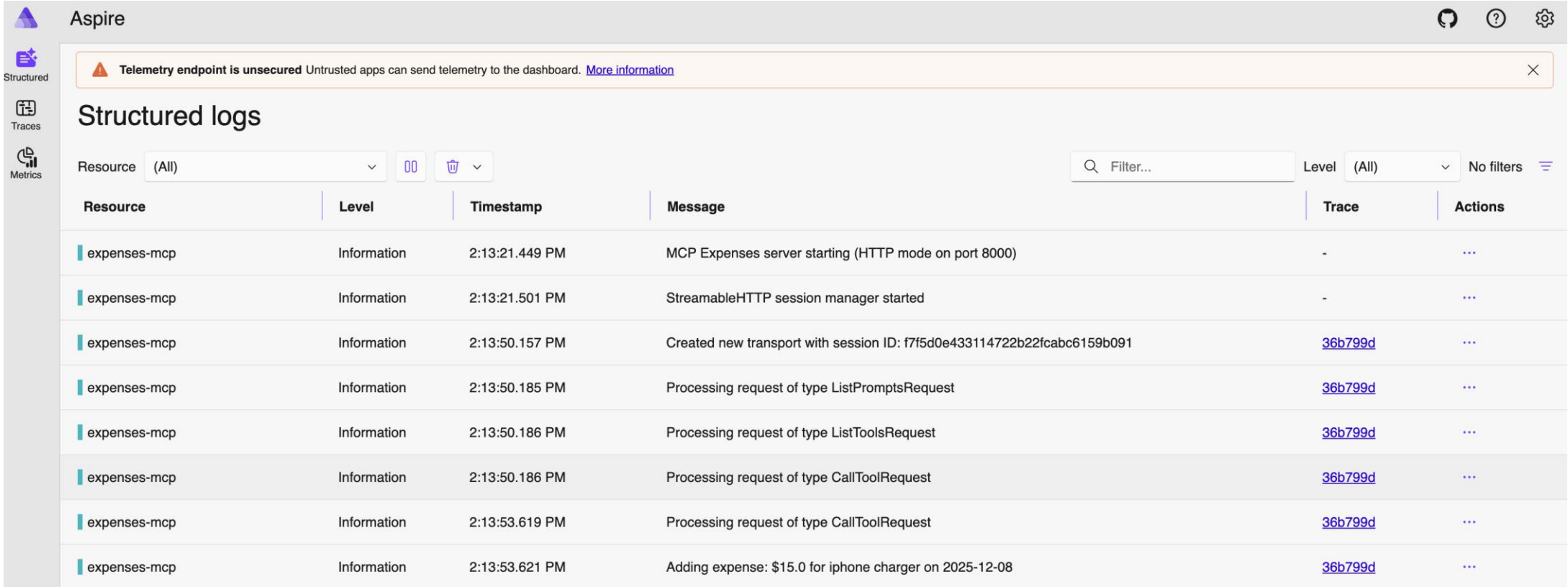
```
npx @modelcontextprotocol/inspector
```

Tip: MCP inspector works best locally, due to CORS issues in GitHub Codespaces.

github.com/modelcontextprotocol/inspector

Observability with Aspire dashboard

```
docker run --rm -d -p 18888:18888 -p 4317:18889 --name aspire-dashboard \
    mcr.microsoft.com/dotnet/aspire-dashboard:latest
```



Learn more about observability in tomorrow's session on deploying production MCP servers.

Using AI Agents with MCP servers

Agent-framework with local MCP Server

Microsoft agent-framework is a new AI agent framework, the successor to AutoGen and Semantic Kernel, and includes support for MCP servers.

```
with MCPStreamableHTTPTool(
        name="Expenses MCP server",
        url="http://localhost:8000/mcp") as mcp_server,
      ChatAgent(
        chat_client=client,
        name="Expenses Agent",
        instructions="You help users log expenses") as agent):
  query = "yesterday I bought a laptop for $1200 using my visa."
  result = await agent.run(query, tools=mcp_server)
```

[Full example: agentframework_http.py]

Langchain v1 with local MCP server

Langchain v1 is the latest version of langchain that centers around agents and tools.

```
client = MultiServerMCPClient({
    "expenses": {
    "url": "http://localhost:8000/mcp",
    "transport": "streamable_http"}})
tools = await client.get_tools()
agent = create_agent(base_model, tools)

today = datetime.now().strftime("%Y-%m-%d")
user_query = "yesterday I bought a laptop for $1200 using my visa."
response = await agent.ainvoke({
  "messages": [SystemMessage(content=f"Today's date is {today}."),
               HumanMessage(content=user_query)]})
```

Full example: langchainv1_http.py

Next steps

Watch past recordings:

aka.ms/pythonmcp/resources

Come to office hours after each session in Discord:

aka.ms/pythonai/oh

Learn from MCP for Beginners:

aka.ms/mcp-for-beginners

🛠️ Dec 16:

Building MCP servers with FastMCP

☁️ Dec 17:

Deploying MCP servers to the cloud

🔐 Dec 18:

Authentication for MCP servers